## Problem Solving by Searching(Informed Search)

## Practical No.3(A)

**Aim:-** Given an initial configuration of the 8-puzzle and a goal configuration, write a Python program to find the solution using Greedy Best-First Search with the Manhattan Distance heuristic.

### Code:-

```
import heapq

GOAL_STATE = (1, 2, 3,
              4, 5, 6,
              7, 8, 0)

GOAL_POSITIONS = {tile: (i // 3, i % 3) for i, tile in enumerate(GOAL_STATE)}


def get_manhattan_distance(state):
    distance = 0
    for i, tile in enumerate(state):
        if tile != 0:  # Skip the blank tile
            current_row, current_col = i // 3, i % 3
            goal_row, goal_col = GOAL_POSITIONS[tile]
            distance += abs(current_row - goal_row) + abs(current_col - goal_col)
    return distance


def get_neighbors(state):
    neighbors = []
    blank_idx = state.index(0)
    row, col = blank_idx // 3, blank_idx % 3

    moves = [(-1, 0), (1, 0), (0, -1), (0, 1)]  # Up, Down, Left, Right

    for dr, dc in moves:
        new_row, new_col = row + dr, col + dc
        if 0 <= new_row < 3 and 0 <= new_col < 3:
            new_blank_idx = new_row * 3 + new_col

            # Convert to list to swap tiles, then back to tuple
```

```
            new_state = list(state)
            new_state[blank_idx], new_state[new_blank_idx] = new_state[new_blank_idx],
new_state[blank_idx]
            neighbors.append(tuple(new_state))

    return neighbors


def greedy_best_first_search(start_state):
    open_list = []
    heapq.heappush(open_list, (get_manhattan_distance(start_state), start_state, [start_state]))

    visited = set()
    visited.add(start_state)

    while open_list:
        _, current_state, path = heapq.heappop(open_list)

        # Check if goal is reached
        if current_state == GOAL_STATE:
            return path

        for neighbor in get_neighbors(current_state):
            if neighbor not in visited:
                visited.add(neighbor)
                h_value = get_manhattan_distance(neighbor)
                heapq.heappush(open_list, (h_value, neighbor, path + [neighbor]))

    return None


def print_state(state):
    for i in range(0, 9, 3):
        print(list(state[i:i + 3]))
    print()


if __name__ == "__main__":
    initial_configuration = (1, 2, 3,
                             0, 4, 6,
                             7, 5, 8)

    print("Initial State:")
    print_state(initial_configuration)
```

```
solution_path = greedy_best_first_search(initial_configuration)

if solution_path:
    print(f"Solution found in {len(solution_path) - 1} moves!\n")

    for step, state in enumerate(solution_path):
        print(f"Step {step}:")
        print_state(state)
else:
    print("No solution could be found.")
```

## Output:-

```
Initial State:
[1, 2, 3]
[0, 4, 6]
[7, 5, 8]

Solution found in 3 moves!

Step 0:
[1, 2, 3]
[0, 4, 6]
[7, 5, 8]

Step 1:
[1, 2, 3]
[4, 0, 6]
[7, 5, 8]

Step 2:
[1, 2, 3]
[4, 5, 6]
[7, 0, 8]

Step 3:
[1, 2, 3]
[4, 5, 6]
[7, 8, 0]
```

## Practical No.3(B)

**Aim:-** Given a weighted graph representing citie and distances between them. Write a python program to find the shortest path from Arad to Bucharest using A* search with heuristic function(straight-line distance to destination).

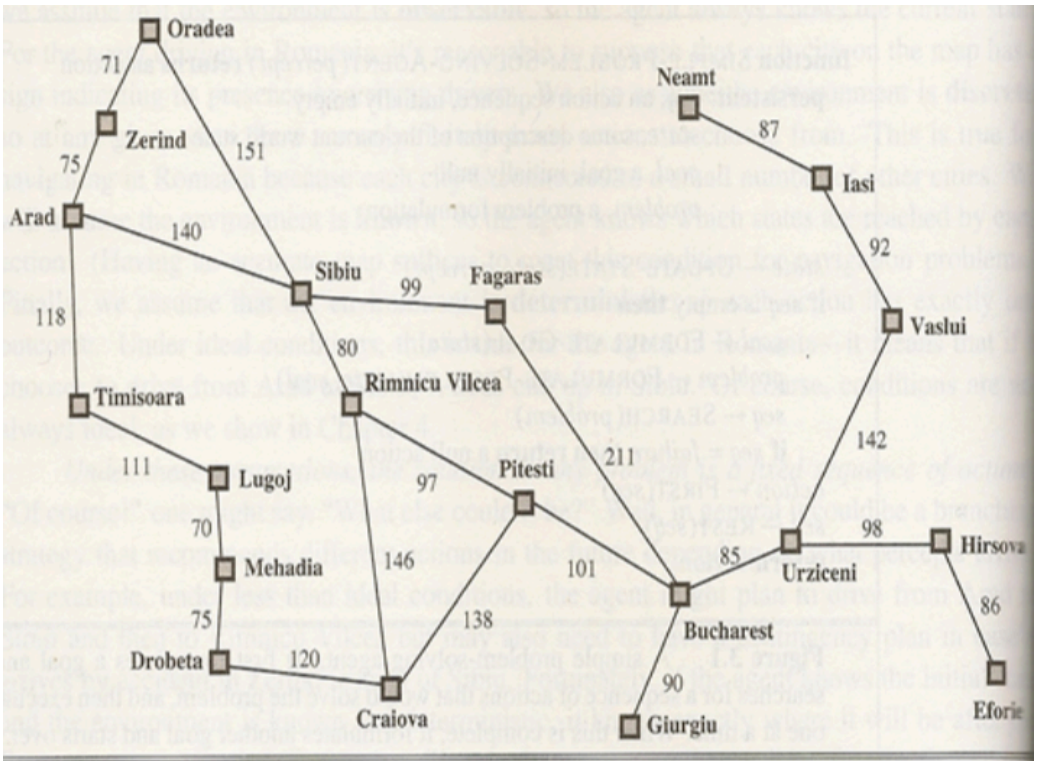



Figure 3.2 A simplified road map of part of Romania.

## Code:-

```
import heapq

# 1. Graph representation extracted directly from the map image:
romania_map = {
    'Arad': {'Zerind': 75, 'Sibiu': 140, 'Timisoara': 118},
    'Zerind': {'Arad': 75, 'Oradea': 71},
    'Oradea': {'Zerind': 71, 'Sibiu': 151},
    'Sibiu': {'Arad': 140, 'Oradea': 151, 'Fagaras': 99, 'Rimnicu Vilcea': 80},
    'Timisoara': {'Arad': 118, 'Lugoj': 111},
    'Lugoj': {'Timisoara': 111, 'Mehadia': 70},
    'Mehadia': {'Lugoj': 70, 'Drobeta': 75},
    'Drobeta': {'Mehadia': 75, 'Craiova': 120},
    'Craiova': {'Drobeta': 120, 'Rimnicu Vilcea': 146, 'Pitesti': 138},
    'Rimnicu Vilcea': {'Sibiu': 80, 'Craiova': 146, 'Pitesti': 97},
```

```
    'Fagaras': {'Sibiu': 99, 'Bucharest': 211},
    'Pitesti': {'Rimnicu Vilcea': 97, 'Craiova': 138, 'Bucharest': 101},
    'Bucharest': {'Fagaras': 211, 'Pitesti': 101, 'Giurgiu': 90, 'Urziceni': 85},
    'Giurgiu': {'Bucharest': 90},
    'Urziceni': {'Bucharest': 85, 'Vaslui': 142, 'Hirsova': 98},
    'Hirsova': {'Urziceni': 98, 'Eforie': 86},
    'Eforie': {'Hirsova': 86},
    'Vaslui': {'Urziceni': 142, 'Iasi': 92},
    'Iasi': {'Vaslui': 92, 'Neamt': 87},
    'Neamt': {'Iasi': 87}
}


# 2. Straight-line distance heuristic h(n) to Bucharest
straight_line_to_bucharest = {
    'Arad': 366, 'Bucharest': 0, 'Craiova': 160, 'Drobeta': 242, 'Eforie': 161,
    'Fagaras': 176, 'Giurgiu': 77, 'Hirsova': 151, 'Iasi': 226, 'Lugoj': 244,
    'Mehadia': 241, 'Neamt': 234, 'Oradea': 380, 'Pitesti': 100, 'Rimnicu Vilcea': 193,
    'Sibiu': 253, 'Timisoara': 329, 'Urziceni': 80, 'Vaslui': 199, 'Zerind': 374
}



def a_star_search(graph, heuristics, start, goal):
    """Solves the shortest path problem using A* Search."""
    # Priority queue stores tuples of: (f_score, counter, g_score, current_city, path)
    counter = 0
    pq = []

    g_score = {start: 0}
    f_score = heuristics[start]

    # Push initial node
    heapq.heappush(pq, (f_score, counter, 0, start, [start]))

    while pq:
        f, _, current_g, current_city, path = heapq.heappop(pq)

        # If goal is reached, return the path and distance
        if current_city == goal:
            return path, current_g

        # Explore neighbors
        for neighbor, distance in graph[current_city].items():
```

```
            tentative_g = current_g + distance

            # If a shorter path to neighbor is found
            if neighbor not in g_score or tentative_g < g_score[neighbor]:
                g_score[neighbor] = tentative_g
                h = heuristics[neighbor]
                f_neighbor = tentative_g + h

                counter += 1
                heapq.heappush(pq, (f_neighbor, counter, tentative_g, neighbor, path + [neighbor]))

    return None, float('inf')

# --- Run Search ---
if __name__ == "__main__":
    start = 'Arad'
    goal = 'Bucharest'

    path, distance = a_star_search(romania_map, straight_line_to_bucharest, start, goal)

    print(f"Shortest Path: {' -> '.join(path)}")
    print(f"Total Distance: {distance} km")
```

## Output:-

Shortest Path: Arad -> Sibiu -> Rimnicu Vilcea -> Pitesti -> Bucharest
Total Distance: 418 km

## Practical No.4 (Baye's Rule Application)

**Scenario:-**

A medical test is used to detect a particular disease.
The probability that a randomly selected person has the disease is 1%.
If a person has the disease, the test returns positive with probability 99%.
If a person does not have the disease, the test still returns positive with probability 5% (false positive rate).
A person takes the test and receives a positive result.

**Aim:-** Write a Python program to compute the probability that the person actually hasthe disease given that the test result is positive.

**Code:-**

```
def calculate_disease_probability():

    prob_disease=0.01
    prob_no_disease=0.99

    true_positive_rate=0.99
    false_positive_rate=0.05

    #1.Calculate the total probability of the positive test result P(T+)
prob_positive_test=(true_positive_rate*prob_disease)+(false_positive_rate*prob_no_disease)

    #2.Apply Baye's Theorem to calculate P(D|T+)
    prob_disease_given_positive=(true_positive_rate*prob_disease)/prob_positive_test

    print(f"Total probability of testing positive P(T+):{prob_positive_test:.4f}")
    print(f"Probability of actually having the disease given a positive test P(D|T+):
{prob_disease_given_positive:.4f}")
    print(f"In Percentage:{prob_disease_given_positive * 100:.2f}%")

if __name__=="__main__":
    calculate_disease_probability()
```

**Output:-**

```
Total probability of testing positive P(T+):0.0594
Probability of actually having the disease given a positive test P(D|T+): 0.1667
In Percentage:16.67%
```